

hokuyo_navigation2

ユーザーマニュアル

3D SLAM and Navigation System Application for RSF-X001
Document Version 0.1.0

2026 年 06 月 15 日

目次

1	はじめに	1
2	セットアップ	2
2.1	ROS 2 のインストール	2
2.2	hokuyo_navigation2 のインストール	3
2.2.1	ソースコードの取得とビルド	3
2.2.2	Docker でのセットアップ	5
2.2.2.1	引数の説明	6
2.3	ユーザ依存関連	7
3	GUI アプリケーションの起動	8
3.1	ファイアウォール設定	8
3.1.1	Ubuntu での設定例	8
3.2	GUI サーバの起動	8
3.3	GUI サーバの停止	9
3.4	GUI の説明	9
4	コンフィグファイルの設定	13
4.1	地図作成パラメータ用のコンフィグファイル	13
4.2	走行順序の定義	14
4.3	編集方法	14
5	データの取得	16
5.1	動作の説明	16
5.2	操作手順の説明	18
6	マッピング	21
6.1	マッピングモードの説明	21
6.2	3D 地図の作成	21
6.2.1	p2o 地図の説明	23
6.2.2	lio_raw 地図の説明	23
7	経路設計	25
7.1	経路の形式	25
7.2	3D Viewer での編集方法	26
7.3	3D Viewer の基本操作	26
7.3.1	ファイルの読み込み	26
7.3.2	選択・回転・移動	28
7.3.3	属性設定	28
7.3.4	追加・削除・保存	28
7.3.5	便利機能	31
8	2D 地図への変換	32
8.1	2D 地図への変換方法	32
8.2	平面化の調整	34
8.3	通行経路の再設定	34
A	問い合わせ先	35

図目次

図 1	GUI サーバ起動後の 2 つの端末	9
図 2	メイン画面	10
図 3	Vizanti	11
図 4	3D Viewer	11
図 5	ファイル管理	12
図 6	コンフィグファイルの編集画面	15
図 7	手動操作モード	17
図 8	手動操作モードの終了時の画面	18
図 9	手動操作モードで扱うボタンの位置 1	18
図 10	手動操作モードで扱うボタンの位置 2	19
図 11	バーチャルジョイスティックの設定方法	20
図 12	マッピングモードの選択画面	22
図 13	地図作成の様子	22
図 14	3D 地図と経路の確認	23
図 15	3D Viewer 上で 3D 点群地図と経路を表示	23
図 16	3D Viewer で waypoint を選択した場合の画面	25
図 17	基本編集画面	27
図 18	アンロード	28
図 19	waypoint の追加押下後の画面	29
図 20	waypoint の削除押下後の画面	30
図 21	waypoint の属性	31
図 22	waypoint の一括向き修正	31
図 23	2D 地図への変換画面	33

表目次

表 1 使用ポート一覧	8
表 2 記録対象のトピック一覧	20

1 はじめに

`hokuyo_navigation2` は、ROS 2 の自律走行パッケージ [Nav2](#) を活用した RSF-X001 (以下、RSF) 専用の屋内屋外対応自律走行ソフトウェアです。

RSF の RTK-GNSS の出力と 3D LiDAR による位置を用いて地図作成と経路設計を行い、それらを用いた自律走行を可能とします。また、非技術者でも簡単に操作を行うための GUI を用意しているため、センサ 1 台で自律走行ロボットの構築が可能となります。

本稿で説明する `hokuyo_navigation2` のソフトウェアバージョンは 1.0.0 に対応しております。このドキュメントバージョンでは、地図作成、経路設計までの方法の説明を行います。

要求バージョン: Ubuntu 22.04, ROS 2 Humble Hawksbill

2 セットアップ

本章では hokuyo_navigation2 含めて全てのセットアップの方法について説明します。

2.1 ROS 2 のインストール

Ubuntu 22.04 LTS に ROS 2 Humble Hawksbill (以下、humble)をインストールする方法について説明します。humble をインストールするには、以下の手順に従います。

humble のリポジトリを追加します。

```
sudo apt update && sudo apt install curl gnupg lsb-release
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
-o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(source /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

humble をインストールします。

```
sudo apt update && sudo apt install ros-humble-desktop-full
```

humble の環境をセットアップします。

```
source /opt/ros/humble/setup.bash
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

インストールの確認として動作テストを行います。

```
# terminal 1
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_cpp talker
# terminal 2
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_cpp listener
```

環境設定を行います。

```
sudo apt install python3-rosdep python3-colcon-common-extensions
sudo rosdep init
rosdep update
```

Gazebo をインストールします。

```
sudo apt-get install gazebo
sudo apt install ros-humble-gazebo-*
```

rqt のプラグインをインストールします。

```
sudo apt install ros-humble-rqt-*
```

ビルドツールのインストールを行います。

```
sudo apt install python3-colcon-common-extensions python3-rosdep
```

ROS 2 の初期設定を行います。

```
sudo rosdep init  
rosdep update
```

ROS 2 ワークスペースの作成を行います。

```
mkdir -p ~/colcon_ws/src  
cd ~/colcon_ws  
colcon build
```

setup.bash をソースして、ROS 2 の環境を有効にする設定を.bashrc に追加します。

```
echo "source $HOME/colcon_ws/install/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

2.2 hokuyo_navigation2 のインストール

ここでは、ソースコードの取得とビルドについて説明します。また、Docker にも対応しています。

2.2.1 ソースコードの取得とビルド

ターミナル操作を行うツールをインストールします。

```
sudo apt-get install -y tree xdotool wmctrl zenity bc
```

URDF 依存パッケージをインストールします。

```
sudo apt-get install ros-tf-transformations ros-humble-joint-state-publisher \  
ros-humble-robot-state-publisher
```

ROS2 パッケージを取得します。

```
cd ~/colcon_ws/src
git clone https://github.com/hokuyo_rsf.git
git clone --recursive https://github.com/Hokuyo-aut/hokuyo_navigation2.git
cd ~/colcon_ws
rosdep update
rosdep install -i --from-path src/hokuyo_navigation2 --ignore-src -r -y
colcon build --symlink-install
```

python パッケージをインストールします。

```
cd ~/colcon_ws/src/hokuyo_navigation2
pip install -r requirements.txt
```

サンプルを動作させるためのモータドライバをインストールします。

```
# yp-spur をインストールします。
mkdir ~/hokuyo_lib
cd ~/hokuyo_lib
sudo apt-get install libmodbus-dev
git clone https://github.com/hokuyo-rd-release/yp-spur.git
cd yp-spur
mkdir build
cd build
cmake ..
make
sudo make install
# icart_mini_driver をインストールします。
cd ~/colcon_ws/src
git clone https://github.com/hokuyo-rd-release/icart_mini_driver_ros2.git
cd ~/colcon_ws
colcon build --symlink-install --packages-select icart_mini_driver
```

hokuyo_slam_ros2 をインストールします。

```
sudo apt-get install libsqlite3-dev sqlite3 libeigen3-dev qtbase5-dev clang qtcreator
libqt5xmlextras5-dev
# proj のインストール
cd ~/hokuyo_lib
wget https://download.osgeo.org/proj/proj-9.4.1.tar.gz
tar -xvf proj-9.4.1.tar.gz
cd proj-9.4.1
mkdir build
cd build
cmake ..
cmake --build . --target install
# pcl 1.14.1 ビルドに時間がかかります。
cd ~/hokuyo_lib
wget https://github.com/PointCloudLibrary/pcl/releases/download/pcl-1.14.1/source.tar.gz -O
pcl.tar.gz
tar -xvf pcl.tar.gz
cd pcl
cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/opt/pcl .
cmake --build build
sudo cmake --install build
```



```

export CMAKE_PREFIX_PATH=$CMAKE_PREFIX_PATH:/opt/pcl
# hokuyo_slam_ros2 のビルド
cd ~/colcon_ws/src/hokuyo_navigation2/hokuyo_slam_ros2
export CMAKE_PREFIX_PATH=$CMAKE_PREFIX_PATH:/opt/pcl
mkdir build
cmake -Bbuild . && cmake --build build
# colcon build
cd ~/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation2
mkdir map/ rosbag/ waypoints/
cd ~/colcon_ws
colcon build
colcon build --symlink-install --packages-select hokuyo_navigation2 lio_nav2_bringup
simple_fastlio_localization

```

GUIを使用するためには、後述の「GUI サーバの起動」を行う前に、ファイアーウォールの設定でポート 5050 を開放する必要があります。ファイアーウォールの設定の詳細は [3 GUI アプリケーションの起動 表 1](#) を参照してください。

2.2.2 Docker でのセットアップ

Docker でのセットアップ方法について記載します。

動作環境: Ubuntu 22.04 LTS, WSL2(Ubuntu 26.04 LTS)

Docker のインストールをします。

```

# Ubuntu 24.04
sudo apt-get install apt-transport-https ca-certificates curl gnupg-agent \
software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
sudo apt-key fingerprint 0EBFCD88
sudo add-apt-repository \
  "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) \
  stable"
sudo apt-get install docker-ce docker-ce-cli containerd.io

# Ubuntu 26.04
sudo apt-get install apt-transport-https ca-certificates curl gnupg-agent \
software-properties-common
sudo chmod 0755 /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/
keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://
download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt install docker-ce docker-ce-cli containerd.io

```

Docker コマンドの権限を取得します。

```

sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker

```

```
# bash-completion
sudo apt-get install bash-completion
source /etc/bash_completion
rm /etc/apt/apt.conf.d/docker-clean
sudo apt-get update
```

Dockerfile のクローン :hokuyo_rsf センサドライバのみビルド済みのイメージをビルドする場合、以下のリポジトリをクローンし Docker ファイルがあるディレクトリへ移動します。

```
cd <YOUR_ROS2_WS>
git clone https://github.com/Hokuyo-aut/hokuyo_rsf.git
cd hokuyo_rsf/docker
```

Dockerfile のクローン :ナビゲーションソフトウェアすべてビルド済みのイメージをビルドする場合、以下のリポジトリをクローンし Docker ファイルがあるディレクトリへ移動します。

```
cd <YOUR_ROS2_WS>
git clone https://github.com/hokuyo-rd-release/hokuyo_navigation2.git
cd hokuyo_navigation2/docker
```

Docker イメージのビルドを行います。提供されている Dockerfile を使用してイメージを作成します。通信エラーを防ぐため `--network host` オプションを付けて実行します。

```
docker build --network host -t hokuyo_navigation2:release .
```

ポイント: `-t` オプションでイメージ名(タグ)を `hokuyo_navigation2:release` として指定しています。(イメージ名は何でも構いませんが、以降のコマンドではこのイメージ名を使用します。)

コンテナの作成と実行を行います。作成したイメージを元にコンテナを起動します。このリポジトリには、 便利な起動用スクリプト `run.bash` が用意されています。

```
chmod +x run.bash
./run.bash -n hokuyo_navigation2 -s /path/to/your/ros2_ws
```

2.2.2.1 引数の説明

-n: 使用する Docker イメージ名を指定します。

-s: ホスト側のディレクトリをコンテナ内と共有するためのパス(絶対パス)を指定します。ソースコードの編集などをホスト側で行いたい場合に便利です。

コンテナ作成後は、`exit` してコンテナの外に出ると、`home` ディレクトリに `CONTAINER_NAME.bash` (`CONTAINER_NAME` は、自分で作成したコンテナの名前)が生成されます。

```
cd
./CONTAINER_NAME.bash
```

次回からは上記のスクリプトを実行すると自動でコンテナをスタートしてコンテナ内に入ることができます。

2.3 ユーザ依存関連

上述のシステム構成において、お客様にてご準備頂くモータドライバの設定方法について説明します。モータドライバはROS 2 トピックで制御されることを前提としており、サンプルでは [こちらのシェルスクリプト nav_common.sh](#) を通じて ROS 2 モータドライバの起動コマンドを実行しております。launch_motor_driver 関数で呼び出すモータのコマンドを変更して、hokuyo_navigation2 パッケージを再度ビルドしてください。

```
# モータドライバを起動する関数
launch_motor_driver() {
  if [ "${use_motor_driver}" = "true" ]; then
    echo "モータドライバを起動します..."
    gnome-terminal -- bash -c "ros2 launch hokuyo_navigation2 icart_mini_drive_launch.xml; bash"
    echo "モータドライバ関連ノードの起動を待っています..."
    echo "モータドライバ関連ノードの起動を確認しました。"
  fi
}
```

3 GUI アプリケーションの起動

本章では、GUI アプリケーションの起動方法について説明します。GUI アプリケーションは、ロボットの状態を可視化し、経路設計や地図の編集などの操作を行うためのツールです。GUI アプリケーションを使用する前に、ファイアーウォールの設定を行い、GUI サーバを起動する必要があります。

3.1 ファイアーウォール設定

GUI アプリケーションは、通信するために特定のポートを使用します。ファイアーウォールの設定を行い、[表 1](#) のポートを開放してください。

ポート番号	プロトコル	用途
5000, 5001	TCP	GUI サーバー / API 通信
5050	TCP	GUI メタデータ管理
9090	TCP	rosbridge (Vizanti 通信用)
10940	TCP	システム内部通信
7400–7800	UDP	ROS 2 (DDS) Discovery 通信

表 1: 使用ポート一覧

3.1.1 Ubuntu での設定例

Ubuntu 標準のファイアーウォール管理ツール `ufw` を使用する場合、以下のコマンドを実行して設定を適用します。

```
# 各種TCPポートの開放
sudo ufw allow 5000,5001,5050,8000,9000,9090,10940/tcp

# ROS 2通信用UDPポートの開放
sudo ufw allow 7400:7800/udp

# 設定の有効化と状態確認
sudo ufw enable
sudo ufw status
```

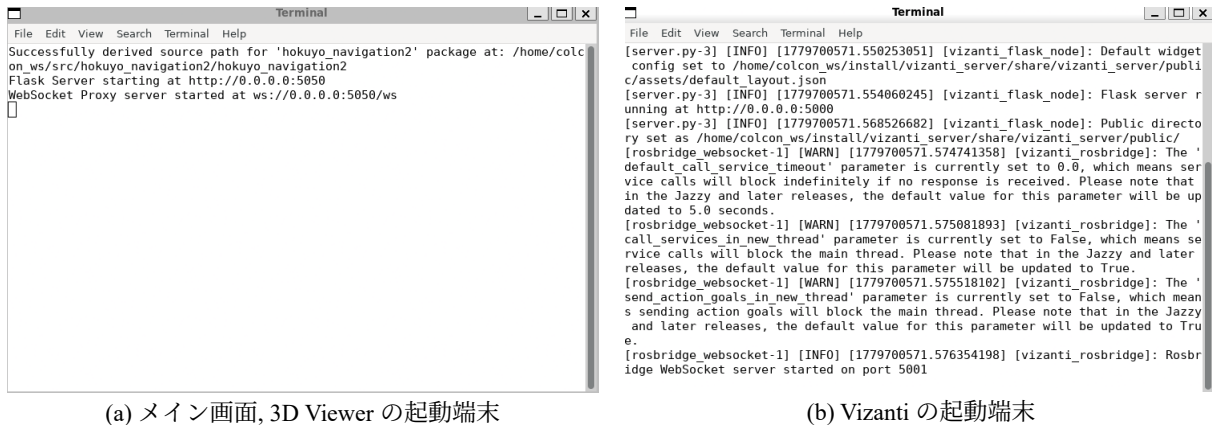
※ ネットワーク環境（社内 LAN など）によっては、ルータや上位のファイアーウォール側で同様の許可設定が必要になる場合があります。詳細な手順については、ネットワーク管理者に問い合わせるか、オペレーティングシステムのドキュメントを参照してください。

3.2 GUI サーバの起動

以下のコマンドで GUI サーバを起動します。

```
cd ~/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation2/scripts
./start_server.sh
```

起動が成功すると [図 1](#) のような 2 つの端末が開きます。これらはすぐに最小化されます。



(a) メイン画面, 3D Viewer の起動端末

(b) Vizanti の起動端末

図 1: GUI サーバ起動後の 2 つの端末

サーバの起動後、下記リンクをブラウザで開いて、GUI アプリケーションにアクセスしてください。

- <http://localhost:5050>

3.3 GUI サーバの停止

以下のコマンドで GUI サーバを停止します。

```
cd ~/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation2/scripts
./stop_server.sh
```

3.4 GUI の説明

GUI アプリケーション画面は、主にメイン画面、3D Viewer、Vizanti、ファイル管理の 4 つで構成されています。

メイン画面: メインの操作画面で、図 2 に示します。ROS ノードの起動、3D Viewer の表示、「自律走行経路確認 Viewer」(以降 Vizanti と呼ぶ)、各種設定や状態の確認が可能です。メイン画面の操作方法に関しては、4 コンフィグファイルの設定 や、5 データの取得、6 マッピング や 8 2D 地図への変換等を参照してください。



図 2: メイン画面

- 現在のモード：アプリケーションの起動状態を表示します。「現在のモード」は下記 3 種類あります。
- 停止モード：デフォルトの状態、ROS ノードやモータドライバは起動していません。
- 手動操作モード：ROS ノードとモータドライバが起動しており、ROS トピック (/wizurg/cmd_vel) を介して手動でロボットを操作できる状態です。
- サーバ接続なし：サーバが起動していない状態で、GUI アプリケーションがサーバに接続できない場合に表示されます。サーバを再起動すると、「停止モード」に戻ります。

Vizanti: 図 2 に示す、「自律走行経路確認 Viewer」をクリックすると、図 3 に示す、「Vizanti」画面が表示されます。「Vizanti」は、ROS 1/ROS 2 のトピックをウェブ上で可視化するためのツールです。2D 地図/経路(以下 waypoint と呼ぶ)の表示、rosviz の取得などが可能です。

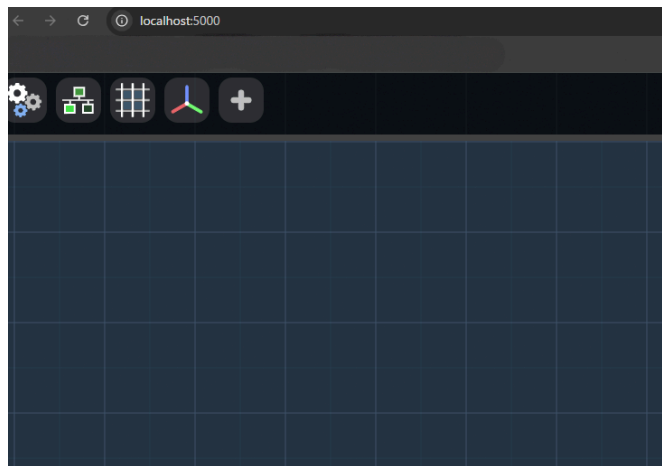


図 3: Vizanti

3D Viewer: 図 2 の「Map Viewer」をクリックすると、図 4 に示す「3D Viewer」が表示されます。「3D Viewer」は、地図、2D 地図の表示と waypoint の編集をするためのツールです。3 次元上に 3D 地図・2D 地図・2D 経路(waypoint)を重ねて表示可能です。表示可能なファイル形式は、pcd, pgm(yaml 必須), json 形式の waypoint のみです。

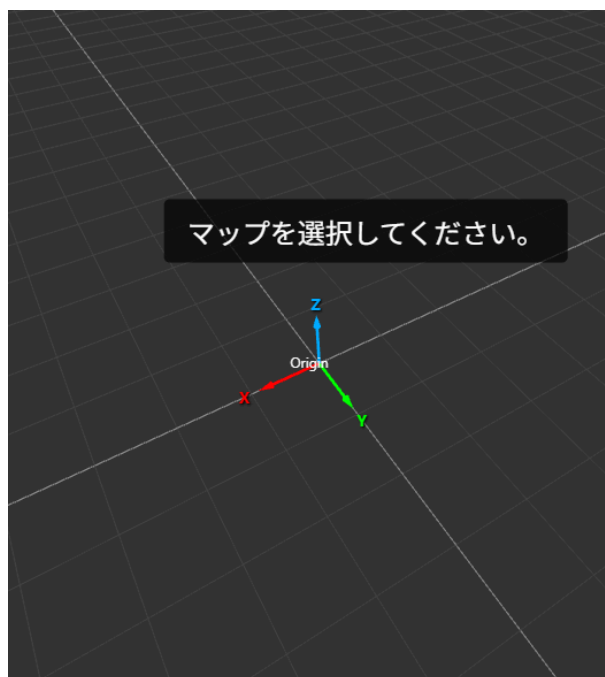


図 4: 3D Viewer

ファイル管理: 過去に作成したデータの管理を行う画面です。不要になった古い地図や waypoint を削除したり、名前を変更したりできます。詳細に関しては、[4 コンフィグファイルの設定](#) を参照してください。

ファイル管理

管理したいファイルの種類を選択してください。

マップ管理

ウェイポイント管理

設定ファイル管理

閉じる

図 5: ファイル管理

4 コンフィグファイルの設定

コンフィグファイルは、地図作成等の動作に必要なパラメータを定義するためのファイルです。コンフィグファイルの設定は、「地図作成」と「自律走行」の機能を使うために必要な手順です。この操作は、図 5 の「ファイル管理画面」から行います。地図作成を実行する際や、複数地図を切り替えて走行する際に、コンフィグファイルを選択して使用します。コンフィグファイルは複数用意することができ、ユーザは必要に応じて適切なコンフィグファイルを GUI から選択して使用することができます。コンフィグファイルの形式は、CSV で、以下の種類があります。

4.1 地図作成パラメータ用のコンフィグファイル

地図作成コンフィグ: 地図作成・経路設計の動作に必要なパラメータを定義するためのファイルです。CSV 形式で、各行にパラメータ名と値を定義します。¹/hokuyo_navigation2/hokuyo_navigation2/scripts/mapping に保存されているシェルスクリプトが地図作成の実行時にこのコンフィグファイルを読み込んで²、地図作成の動作に必要なパラメータ名は option_name であり、value の列を編集して設定します。option_name について下記で解説します。default³は、パラメータの参考値として記載しています。

```
option_name,value,default
gnss_topic,/fix,/fix
pointcloud_topic,/hokuyo3d/hokuyo_cloud2,/hokuyo3d3/hokuyo_cloud2
lio_topic,/hokuyo_lio/lidar_odom,/hokuyo_lio/sync_odom
run_lio,true,true
gnss_cov_thre,0.01,0.01
imu_topic,/hokuyo3d/imu
slam_mode,gnss
pc_save_distance,1.0
wp_save_distance,2.0
gnss_min_movement_thre,4.0
lio_min_movement_thre,0.1
gravity_stride,1
orig_frame,yvt,yvt
target_frame,lio_odom,lio_odom
thre_z_min,-1.0,-1.0
thre_z_max,20.0,20.0
map_resolution,0.05,0.05
thres_point_count,1,1
flag_pass_through,False,False
thre_radius,0.1,0.1
waypoint_tolerance,1.0,1.0
fix_rate,40,40
```

gnss_topic: 地図作成で用いる GNSS トピックの名前を定義します。(GNSS 補正)

pointcloud_topic: 地図作成で用いる pointcloud2 トピックの名前を定義します。(p2o,lio_raw)⁴

lio_topic: 地図作成で用いる LIO(Odometry)トピックの名前を定義します。(p2o,lio_raw)

run_lio: 本パッケージでは使いません。(RSF 以外の LIO を扱う際等の拡張機能として活用ください。)

gnss_cov_thre: GNSS 補正の地図作成に用いる GNSS(navsatfix)トピックの分散の閾値を定義します。(p2o)

imu_topic: IMU トピックの名前を定義します。(p2o)

slam_mode: 地図作成モードを定義します。(gravity: IMU のみでの地図作成、その他文字列: GNSS 補正による地図作成)(p2o)

pc_save_distance: pointcloud の保存間隔 [m] を定義します。(p2o, lio_raw)

wp_save_distance: 経路の保存距離 [m] を定義します。(p2o, lio_raw)

¹CSV ファイル内に半角スペース（カンマの前後など）が含まれないように注意してください。正常に読み込まれない原因となります。

²地図作成コンフィグファイルは、地図作成の実行時に読み込まれます。地図作成の実行前に、適切な値に設定して保存してください。地図作成の実行中にコンフィグファイルを編集しても、地図作成の動作には反映されません。

³default の値は、あくまで参考値であり、実際の環境や使用するセンサに応じて適切な値に設定してください。例えば、gnss_cov_thre は、GNSS の受信状況や精度に応じて調整が必要です。

⁴pointcloud_topic と lio_topic の周期は同じ（例：共に 20Hz）ものを指定してください。周期が異なる場合、地図作成の精度が低下する可能性があります。

gnss_min_movement_thre: GNSS の最小移動閾値を定義します。(p2o)

lio_min_movement_thre: LIO の最小移動閾値を定義します。(p2o)

gravity_stride: オドメトリの間引き間隔(指定した回数に 1 個のデータを使用)を定義します。(p2o)

orig_frame: RSF のフレームの名前を定義します。(lio_raw)

target_frame: LIO のフレームの名前を定義します。(lio_raw)

thre_z_min: 3D 点群を 2D 化する際に、扱う点群データの鉛直方向の高さの閾値の最小値 (2D 化)

thre_z_max: 3D 点群を 2D 化する際に、扱う点群データの鉛直方向の高さの閾値の最大値 (2D 化)

map_resolution: 2D 地図の解像度 [m] を定義します。(2D 化)

thres_point_count: 指定した半径内に thres_point_count 以上の点がない場合、その点は「孤立したノイズ (外れ値)」とみなされ、削除されます。値を大きくすると、ノイズに強くなりますが、細いポールやワイヤーなどの薄い障害物が消えてしまいます。小さくすると、1 点でもあれば障害物とするため、安全側ですが、ノイズによる「行けない場所」が増えやすくなります。(2D 化)

flag_pass_through: ロボットがぶつかる可能性のある「特定の高さにある点」だけを抽出するかどうかを定義します。(2D 化)

thre_radius: 「ノイズ除去」において、近傍点を探すための「検索半径」を意味します。大きくするとより広い範囲で密度を判定します。まばらにちらばったノイズを消しやすくなりますが、細いポールやワイヤーあるいは、物体の角などの「密度が低い本物の物体」まで消えてしまうリスクがあります。値を小さくする、より厳密に近接している点だけを保持します。センサの特性上発生する「浮遊したゴミ(ゴースト)」を除去するのに適していますが、非常に近い距離に固まっているノイズは除去できません。(2D 化)

waypoint_tolerance: waypoint とそれらをつなぐ経路の周囲を、フリースペース (移動可能領域) としてマークするための許容半径 [m] を示しています。(2D 化)

fix_rate: gnss_cov_thre 以下の GNSS トピックの割合を指定します。この割合以上の場合 GNSS を使った補正による地図作成を行います。この割合以下の場合、z 軸を 0 として制約をかけて 3D 点群地図の鉛直方向の反りを軽減するような補正をすることによって 3D 点群地図を作成します。(p2o)

4.2 走行順序の定義

シナリオファイル: 複数の地図を切り替えて走行する際の地図走行順序を定義するためのファイルです。CSV 形式で、各行に地図のファイル名と走行順序を定義します。⁵

```
map_file,waypoint_file,nav_type,interval
map1,waypoint1,loc,10
map2,waypoint2,gnss,30
map3,waypoint3,loc,5
```

map_file: 使用する地図ファイルの名前を定義します。地図ファイルは、3D 点群地図(pcd 形式)と 2D 地図(pgm 形式)を使用します。この際、3D 地図と 2D 地図のファイル名は同じにしてください。例えば、3D 地図が map1.pcd の場合、対応する 2D 地図は map1.pgm になります。

waypoint_file: 使用する経路ファイルの名前を定義します。経路ファイルは、json 形式の waypoint ファイルを使用します。waypoint ファイルの詳細な形式や作成方法については「第 7 経路設計 章」を参照してください。

nav_type: 走行モードを定義します。走行モードは、loc(3D 地図とのマッチング)、gnss(リアルタイム)の 2 種類があります。

interval: 地図の切り替え間隔 [s] を定義します。multi_map 走行モードで複数の地図を切り替える際に使用します。例えば、10 秒ごとに地図を切り替える場合は、interval を 10 に設定します。

コンフィグファイルの編集方法や、各パラメータの説明については以下のセクションで詳しく説明します。

4.3 編集方法

コンフィグファイルは、「ファイル管理」の「設定ファイル管理」からブラウザ上で編集できます。ブラウザ上での画面を 図 6 (a) ～ 図 6 (b) に示します。

地図作成コンフィグの編集: 図 6 (a) の「テキスト編集」ボタンをクリックすると、図 6 (b) の画面が表示

⁵CSV ファイル内に半角スペース (カンマの前後など) が含まれないように注意してください。正常に読み込まれない原因となります。

示され、コンフィグファイルの内容がテキストエリアに表示されます。テキストエリア内で内容を編集し、「保存」ボタンをクリックすると、変更が保存されます。編集の際は、CSV 形式を維持するように注意してください。例えば、パラメータ名と値の間はカンマで区切り、各行は改行で区切る必要があります。また、CSV ファイル内に半角スペース（カンマの前後など）が含まれないようにしてください。正常に読み込まれない原因となります。

シナリオファイルの編集: シナリオファイルも、地図作成コンフィグと同様の方法で編集できます。図 6 (a) の「テキスト編集」ボタンをクリックし、テキストエリア内で内容を編集して保存してください。シナリオファイルも CSV 形式を維持するように注意してください。

- シナリオファイルに関しては、図 6 (a) の「編集」ボタンをクリックした後に、図 6 (c) の画面が表示されます。これは専用のシナリオ編集画面で、シナリオファイルの内容を表形式で編集できるようになっています。表形式の編集画面では、地図ファイル名、経路ファイル名、走行モードをそれぞれの列に入力して編集できます。切り替え間隔は数値を入力します。行の追加・削除は、ボタンをクリックすると即座に実行されます。また、各行の左に、“三”の字のアイコンを画面の上下にドラッグアンドドロップすることで、行ごとの入れ替えが可能です。編集後は、「保存」ボタンをクリックして変更を保存してください。

ファイル名の編集: コンフィグファイルのファイル名を変更したい場合は、図 6 (a) の編集したいコンフィグファイルの文字列をダブルクリックすると、ファイル名を編集できるようになります。新しいファイル名を入力し、選択解除すると、ファイル名の変更が反映されます。



(a) 設定ファイル管理

テキスト編集: hokuyo_slam_t

オプション, 指定値, デフォルト値

```
gnss_topic, /fix, /fix
pointcloud_topic, /hokuyo3d/hokuyo_cloud2, /hokuyo3d3/hokuyo_cloud2
lio_topic, /rsf/lio_lidar_rate_odom, /hokuyo_lio/sync_odom
run_lio, true, true
gnss_cov_thre, 0.01, 0.01
imu_topic, /hokuyo3d/imu
slam_mode, gravity
pc_save_distance, 1.0
wp_save_distance, 4.0
gnss_min_movement_thre, 4.0
lio_min_movement_thre, 0.1
gravity_stride, 1
```

(b) テキスト編集

CSV編集: maps_and_waypoints.csv

	Map File	Waypoint File	Nav Type	Interval (sec)	削除
三	toyonaka_test	toyonaka_test	gnss	5	削除
三	toyonaka_test	toyonaka_test	loc	20	削除
三	toyonaka_test	-- 選択 --	-- 選択 --	1	削除

行を追加 保存 キャンセル

(c) シナリオ編集

図 6: コンフィグファイルの編集画面

5 データの取得

図 2 のメイン画面の「データ取得」ボタンをクリックすることで、ROS ノード、モータドライバの起動・停止を行い、Vizanti 画面で地図・経路作成に必要な rosbag を取得します。この手順を実施する前に、RSF のノードのパラメータ(IP アドレスや、rostopic 名、tf フレーム名等)を/hokuyo_navigation2/hokuyo_navigation2/config/rsf_node_config.yaml で確認して下さい。

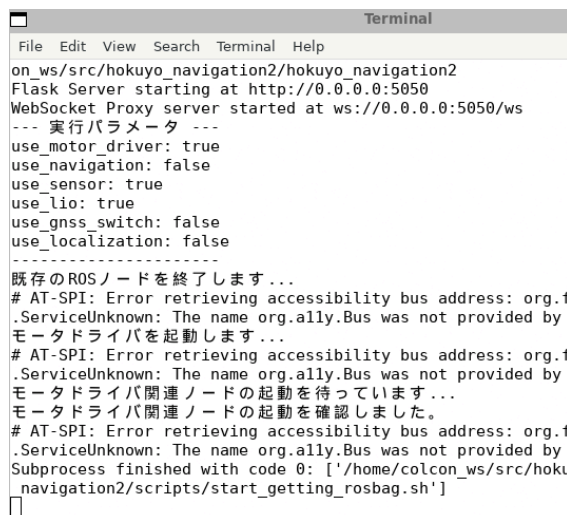
5.1 動作の説明

「データ取得」ボタンをクリックすると、画面が遷移し、「センサデータの記録が開始されました」と表示されます。すると、図 7 に示すターミナルと「ロボットプログラム停止用ポップアップ」⁶が起動し、RSF の ROS ノードとモータドライバが起動します。ターミナルはすぐに最小化されます。GUI はメイン画面に戻り、「現在のモード」が「手動操作モード」に変わります。この際 図 7 (a) のように、メイン画面のボタンの内、誤動作防止のため「データ取得」、「マッピング」、「自律走行」、「ファイル管理」ボタンが無効化され、「ロボット停止」ボタンと「Map Viewer」のみが有効化されます。

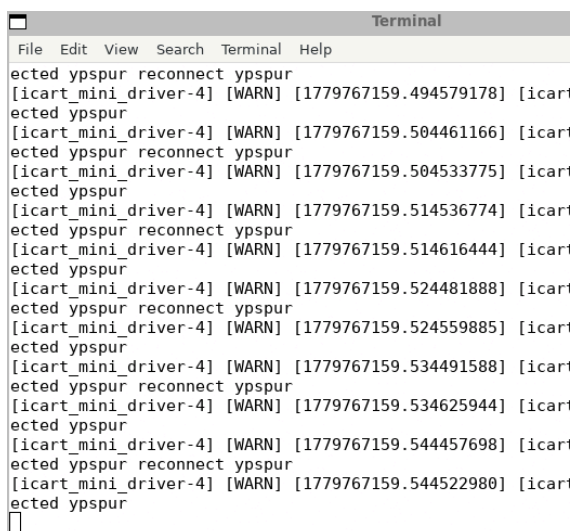
⁶万が一 ROS ノードやモータドライバが起動した状態のままウェブ GUI が無効化された場合は、図 8 (b) のポップアップを使用することで、ROS ノードとモータドライバを停止させてください。



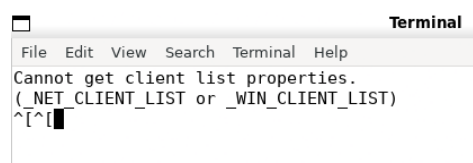
(a) 手動操作モード起動後のメイン画面



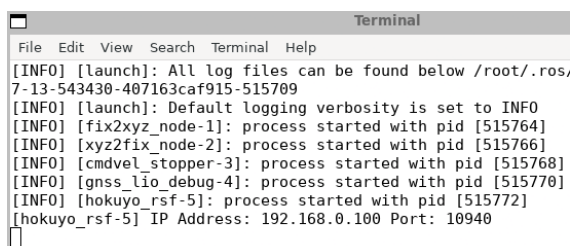
(b) メイン画面の起動ターミナル(モータドライバの起動)



(c) モータドライバの起動ターミナル



(d) その他ターミナル



(e) hokuyo rsf ノードの起動ターミナル

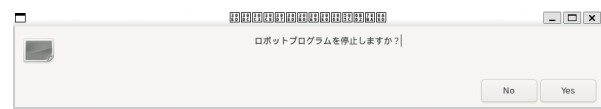
図 7: 手動操作モード

その後、Vizanti 画面を開き、rosvag の記録と バーチャルジョイスティックを介して、ROS トピックでロボットの操縦を行います。rosvag の記録を停止して、ロボットを停止させると「データ取得」の一連の流れを終了します。

```

Terminal
File Edit View Search Terminal Help
既存のROSノードを終了します...
# AT-SPI: Error retrieving accessibility bus address: org.freed
.ServiceUnknown: The name org.ally.Bus was not provided by any
モータドライバを起動します...
# AT-SPI: Error retrieving accessibility bus address: org.freed
.ServiceUnknown: The name org.ally.Bus was not provided by any
モータドライバ関連ノードの起動を待っています...
モータドライバ関連ノードの起動を確認しました。
# AT-SPI: Error retrieving accessibility bus address: org.freed
.ServiceUnknown: The name org.ally.Bus was not provided by any
Subprocess finished with code 0: ['/home/colcon_ws/src/hokuyo_n
_navigation2/scripts/start_getting_rosvag.sh']
--- Emergency Stop initiated by Web Interface ---
Stopping multi-map navigation script...
stopping single-map navigation script...
Stopping rosvag play or record...
Closing rsf process!
Closing bringup launch process!
Closing motor_drive process!
Closing Navigation & Waypoint process!
kill all ros2 nodes!
Subprocess finished with code -15: ['/home/colcon_ws/src/hokuyo_
yo_navigation2/scripts/ctrl/web_kill_all_rosnode.sh']

```



(b) ロボットプログラム停止用ポップアップ

(a) メイン画面の起動ターミナル(ロボット停止後)

図 8: 手動操作モードの終了時の画面

5.2 操作手順の説明

1. メイン画面上の「データ取得」ボタンをクリックすると、画面が遷移し、元の画面で表示が変わります。その後、図 9 (b) の自律走行経路確認 Viewer(Vizanti)を開いてください。



(a) データ取得ボタンの位置

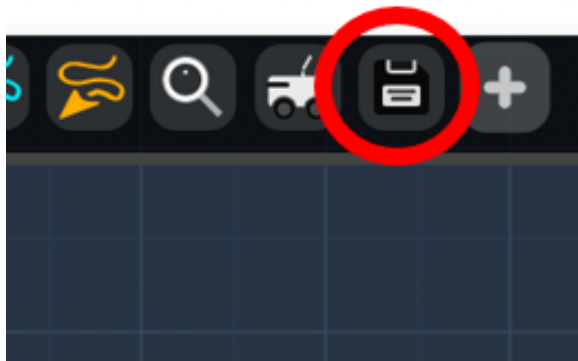


(b) Vizanti の起動ボタンの位置

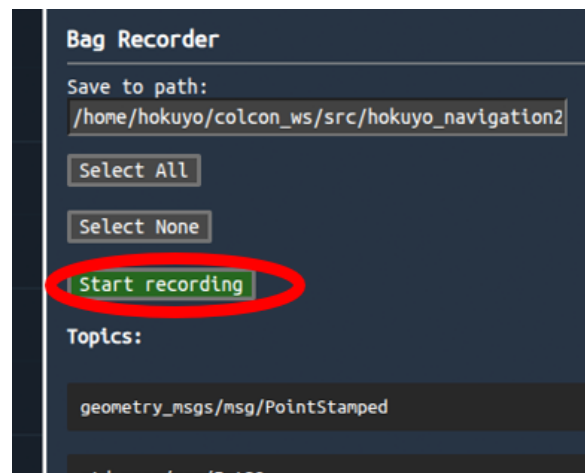
図 9: 手動操作モードで扱うボタンの位置 1

2. 図 10 (a) の rosvag 取得ボタンをクリックして、rosvag の保存名を「Save to path :」に /path/to/hokuyo_navigation2/hokuyo_navigation2/rosvag/<ROS2BAG_NAME> を入力し、(<ROS2BAG_NAME> は ROS2 のためディレクトリ名)「Topics :」に表 2 の rostopic を選択し、図 10 (b) の「start recording」ボタンを

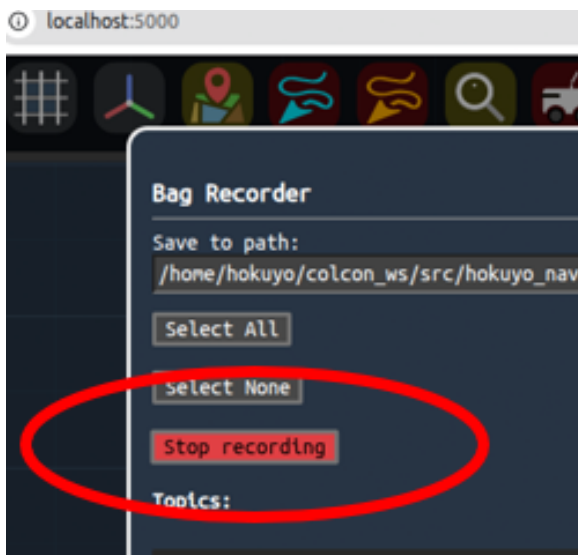
押して下さい。rosvbag 取得ボタンが表示されていない場合は「+」ボタンをクリックし、図 10 (d) の画面を表示し、「Bag Recorder」を表示させて下さい。record ボタンを押すと、recording が開始され、図 10 (d) のようにアイコンの表示が赤に変わります。



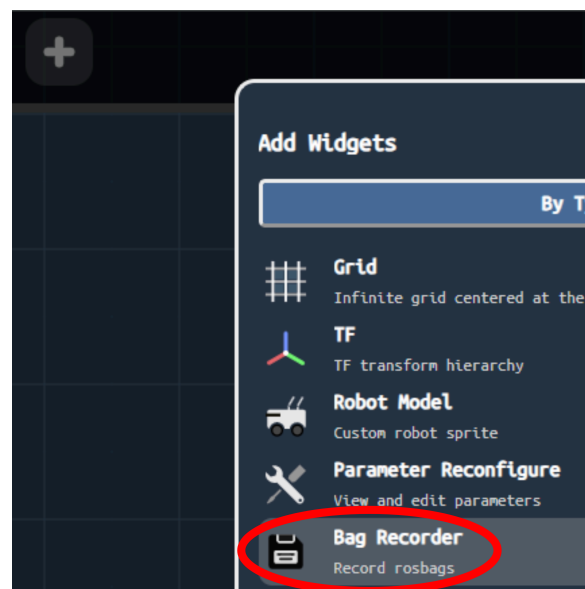
(a) rosvbag 取得ボタン



(b) start recording ボタンの位置



(c) stop recording ボタンの位置



(d) Bag Recorder ボタンの表示

図 10: 手動操作モードで扱うボタンの位置 2

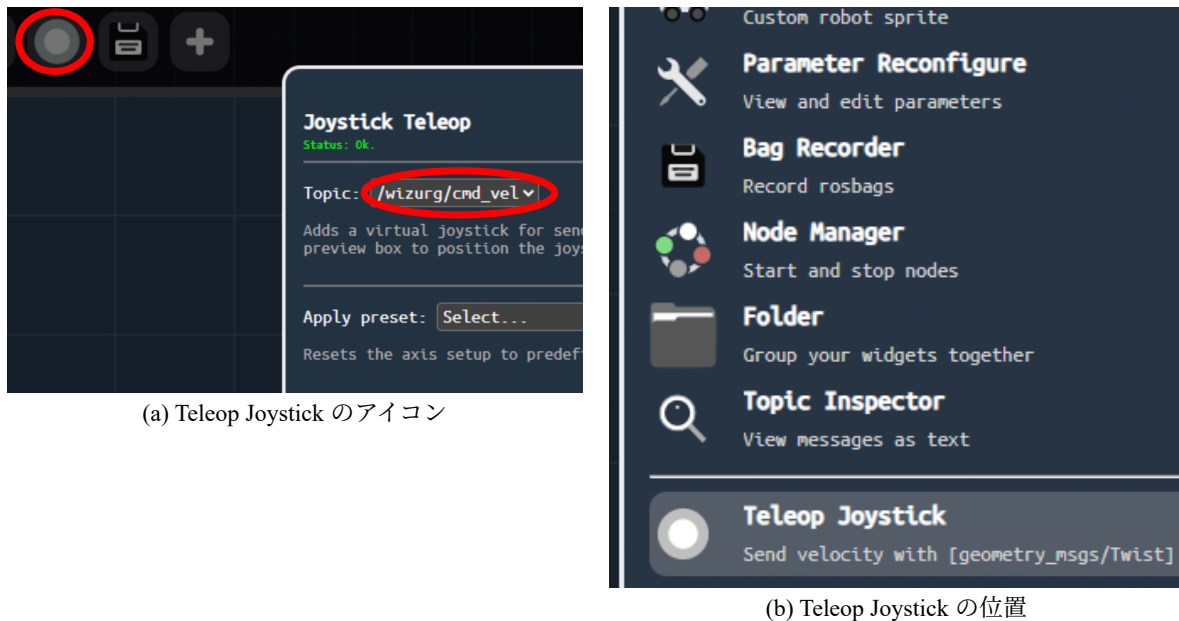


図 11: バーチャルジョイスティックの設定方法

トピック名 ⁷	メッセージ型
/rsf/lio_lidar_rate_odom	nav_msgs/Odometry
/hokuyo3d/imu	sensor_msgs/Imu
/hokuyo3d/hokuyo_cloud2	sensor_msgs/PointCloud2
/fix	sensor_msgs/NavSatFix
/gga	nmea_msgs/Gpgga

表 2: 記録対象のトピック一覧

- rostopic 選択画面外をクリックして Vizanti に戻ってください。
- 図 10 (d) の画面からジョイスティックの「+」アイコンを選択し、Vizanti でバーチャルジョイスティックを表示させてください。
- 図 11 (a) のバーチャルジョイスティックのアイコンを選択して、ロボットの速度トピック名を選択して、Vizanti に戻ると、バーチャルジョイスティックが rostopic をパブリッシュします。速度の機構のいくつかに対応しています。ロボットの足回りに合わせて Apply preset を選択し、加速度、速度の上限・下限を調整して下さい。
- rosviz 取得を実行している間に、バーチャルジョイスティックでロボットを操作して、rosviz の記録作業を行います。
- データ取得が完了したら 図 10 (c) の Vizanti の rosviz 取得アイコンをクリックして stop recording をクリックして rosviz の取得を終了して下さい。
- Vizanti のブラウザタブを閉じて下さい。
- 図 2 「ロボット停止」を押して、RSF の ROS ノードとモータドライバを停止させて下さい。「データ取得」で起動したターミナルが自動で全て閉じられ、GUI サーバのターミナルに、ノードがキルされた文言が表示されます。その後、GUI は 図 2 の現在のモードが「停止モード」となります。

⁷トピック名はデフォルト値を記載しています。

6 マッピング

「マッピング」は [5 データの取得](#) で取得した rosbag をデシリアライズすることで 3D 点群地図(pcd)を作成することを指します。本章ではマッピングのモードと操作手順について説明します。pcd を 2D 占有格子地図(pgm)に変換する手順もマッピングの画面から操作しますが、これは手順の都合上、[8 2D 地図への変換](#) で説明します。

6.1 マッピングモードの説明

マッピングボタンをクリックすると下記 p2o, lio_raw, pcd2pgm の 3 種類のモードが使用できます。p2o に関しては、さらに 3 つのサブモードがあり、GNSS の受信状況に応じて使い分けます。

p2o: グラフベース最適化を用いて 3D 点群地図を作成します。p2o には、下記 3 つのモードがあり、それぞれを使用するために、GNSS の受信状況に応じて使い分けます。rosbag をシリアライズして生データを解析するため高速に実行可能です。他のパラメータも地図作成コンフィグから設定可能です。詳細は、[4 コンフィグファイルの設定](#) と、[表 2](#) を参照して下さい。

- GNSS 補正モード：RTK-GNSS の ROS トピック sensor_msgs/NavSatFix の内、地図作成コンフィグで設定した GNSS の共分散の閾値以下の高精度な緯度・経度・高度データを抽出し、地図を作成します。デフォルトの地図作成モードです。使用するトピックは、[表 2](#) の地図作成コンフィグで指定された、nav_msgs/Odometry, sensor_msgs/PointCloud2, sensor_msgs/NavSatFix の 3 種類です。地図作成コンフィグで slam_mode パラメータを gravity 以外の任意の文字列で設定した際に使用可能です。
- IMU 補正モード：IMU データを用いて LIO を補正することで地図作成するモードです。[4 コンフィグファイルの設定](#) で解説した地図作成コンフィグで slam_mode パラメータを gravity と設定することで使用可能です。使用するトピックは、[表 2](#) の nav_msgs/Odometry, sensor_msgs/PointCloud2, sensor_msgs/Imu の 3 種類です。
- z 軸 0 補正モード：gnss モードの内部の機能です。共分散値の閾値よりも小さい値の fix トピックが少ない、またはない場合、z 軸を 0 として制約をかけて 3D 点群地図の鉛直方向の反りの軽減します。これは GNSS の共分散の閾値以下の fix トピックがパラメータ fix_rate % 以下なら実行されます。

lio_raw: LIO を補正せず、3D 点群を LIO の軌跡に沿って並べて地図を作るモードです。LIO は移動距離に応じて累積誤差が蓄積するため、目安として、100m 以上直進する場合、進行方向に対して鉛直方向の反りが発生する可能性があります。例えば、GNSS が使用できない屋内環境等で、IMU 補正モードと併用して活用してください。[4 コンフィグファイルの設定](#) の地図作成コンフィグで使用されるパラメータの内 pointcloud_topic, lio_topic, pc_save_distance は p2o と共用で、orig_frame, target_frame は本モード固有のパラメータです。

pcd2pgm: p2o, lio_raw で生成した 3D 点群地図を pgm 形式の 2D 占有格子地図に変換します。[4 コンフィグファイルの設定](#) の地図作成コンフィグで使用されるパラメータの内 thre_z_min, thre_z_max, map_resolution, thres_point_count, flag_pass_through, thre_radius を使用します。変換の際に使用する waypoint ファイルを選択すると、waypoint に沿って 2D 地図の経路上の静的障害物を除去することができます。詳細は [8 2D 地図への変換](#) で解説します。

6.2 3D 地図の作成

以下、「p2o」モードを例として説明しますが、「lio_raw」モードでも操作は同じです。

1. [図 2](#) 「マッピング」ボタンをクリックします。
2. [図 12 \(a\)](#) で「p2o」ボタンをクリックします。

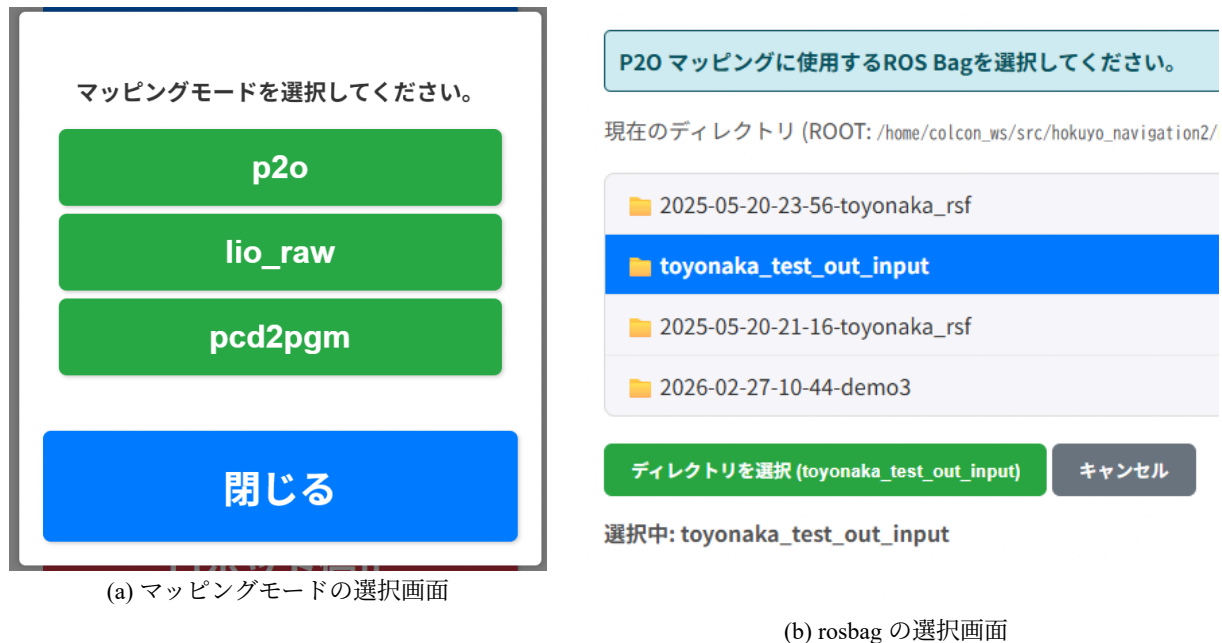


図 12: マッピングモードの選択画面

- 図 12 (b) で地図作成に使用する rosbag ファイルを選択し、「ディレクトリを選択」画面をクリックしてください。
- 図 13 (a) で作成する地図のファイル名を入力し、そのまま実行するか、もしくは、パラメータ設定ファイル(CSV)で作成したパラメータファイルを指定して「p2o マッピングを開始」をクリックすると処理が開始されます。⁸

出力ファイル/マップ名:

toyonaka_test_out_input_processed

P2O マッピング オプション

P2Oモードでは、トピック同期処理後のクリーンなROS Bagを入力されます。

出力マップ名: 上記の「出力ファイル/マップ名」がそのまま最終的に使われます。

パラメータ設定ファイル (CSV):

hokuyo_slam_topics_cfg.csv

P2O マッピングを開始

キャンセル

(a) 地図作成実行画面

```
root@407163caf915: /home/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation
File Edit View Search Terminal Help
step 43: res=0.0924864, convergence_score=1.28269e-07 time=0.024902s
step 44: res=0.0924865, convergence_score=1.11687e-07 time=0.024834s
step 45: res=0.0924866, convergence_score=9.72537e-08 time=0.023631s
step 46: res=0.0924867, convergence_score=8.46884e-08 time=0.022399s
step 47: res=0.0924867, convergence_score=7.3749e-08 time=0.034701s
step 48: res=0.0924868, convergence_score=6.42244e-08 time=0.033895s
step 49: res=0.0924869, convergence_score=5.59312e-08 time=0.024603s
step 50: res=0.0924869, convergence_score=4.87099e-08 time=0.026871s
step 51: res=0.0924869, convergence_score=4.24216e-08 time=0.030163s
converged: -3.76171e-08 < 0.001
data/toyonaka_test_out_input_processed/output.p2o: 1.38191s
Found DB file: /home/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation
toyonaka_test_out_input/toyonaka_test_out_input_0.db3
Processing topic (db): /hokuyo3d/hokuyo_cloud2
Saved 2336 point cloud files (db).
PCDファイルを保存しました: toyonaka_test_out_input_processed_Acord.pcd
情報: Waypointリストの最初2つと最後2つの要素を削除しました。
Waypointファイルを保存しました: /home/colcon_ws/src/hokuyo_navigation2
vigation2/waypoints/toyonaka_test_out_input_processed.json
点群の平行移動を開始します。
点群の平行移動を終了しました。
P2O SLAM completion flag created: /home/colcon_ws/src/hokuyo_navigation
navigation2/map/toyonaka_test_out_input_processed.P2O_DONE
root@407163caf915: /home/colcon_ws/src/hokuyo_navigation2/hokuyo_navigation
```

(b) 実行時のターミナル

図 13: 地図作成の様子

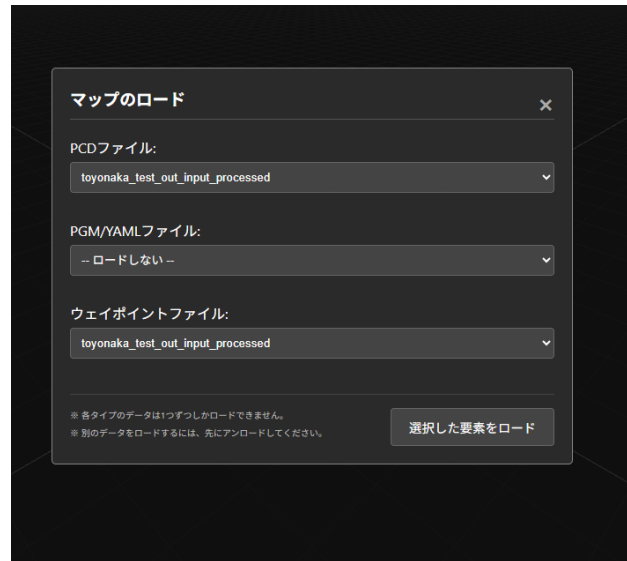
- 図 13 (b) のように表示されたら実行が正常に完了しています。⁹
- 実行後、図 13 (a) の画面が以下のように変わります。「3D ビューワで確認」ボタンをクリックすると、3D Viewer が起動します。3D 地図・経路が自動で作成されているので、図 14 (b) の画面で「PCD ファイル」と「ウェイポイントファイル」からファイルを選択し、「選択した要素をロード」ボタンをクリックすると 3D Viewer でデータを確認することができます。

⁸以前に作成した地図と同じ名前を入力した場合上書きされるため注意が必要です。

⁹実行時間は rosbag のサイズに依存しますが、およそ 30 秒 ~ 1 分程度です。



(a) 地図作成実行後の画面



(b) 3D Viewer で経路と地図を確認

図 14: 3D 地図と経路の確認

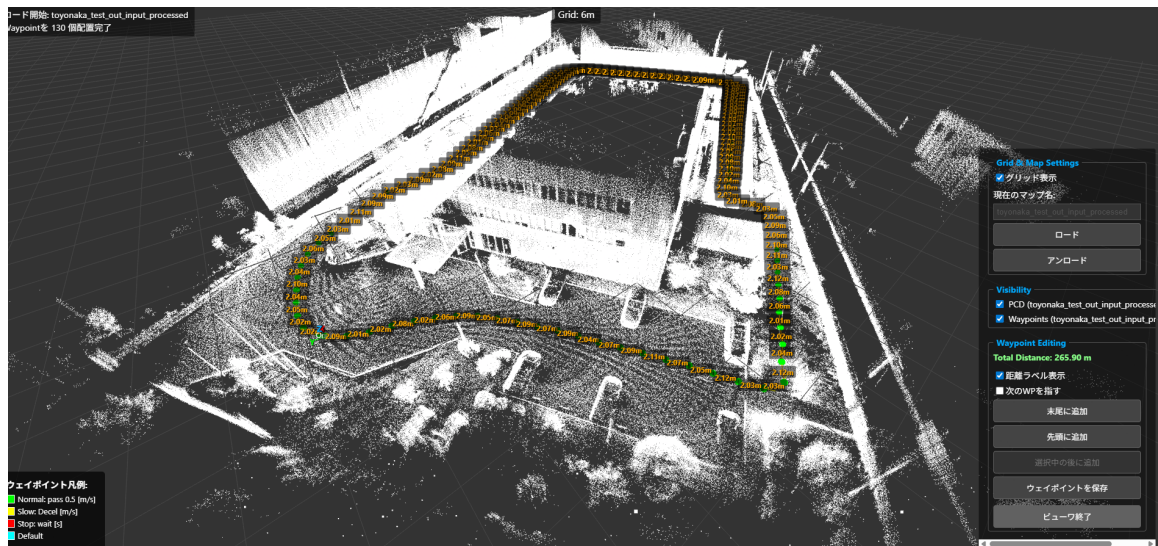


図 15: 3D Viewer 上で 3D 点群地図と経路を表示

1. 実行完了後、図 13 (b) のターミナルと、図 14 (a) のブラウザタブは閉じて下さい。

6.2.1 p2o 地図の説明

p2o により作成された 3D 地図は、UTM 座標系を基準座標が原点 origin となるように並行移動したものです。基準座標とは、その計測エリアの基準となる中心点のことで、この中心点からの相対的なメートル差分として扱われます。並行移動後の地図は、origin を原点としており、3D Viewer 上の x を東、y を北、z を鉛直上方としております。p2o 実行後は、3D 地図のみではなく、計測エリアの基準となる中心点と、緯度・経度・高度の初期位置、初期姿勢(UTM 座標系の x 方向を(0, 0, 0, 1)としたクォータニオン)と初期位置が/hokuyo_navigation2/hokuyo_navigation2/data/<MAP_NAME> に出力されます。

6.2.2 lio_raw 地図の説明

lio_raw により作成された 3D 地図は、オドメトリ座標系を基にした 3D 地図です。センサの起動した位置を原点、その際の x 方向を初期姿勢(0, 0, 0, 1)とします。センサ正面方向を x、センサから見て正面左方向を y、センサ正面から見て鉛直上方を z となります。lio_raw 実行後は、3D 地図のみではなく、こ

これらの初期位置、初期姿勢が/hokuyo_navigation2/hokuyo_navigation2/data/<MAP_NAME> にテキストとして出力されます。

7 経路設計

第6 マッピング 章実施後を想定し、図15の画面の続きから説明します。本章では、マッピングで生成された waypoint の形式の説明と、waypoint の編集を 3D Viewer から行う操作手順を説明します。waypoint とは、ロボットの目的地点のことであり、本システムでは、目標位置、目標姿勢、属性、到着判定の許容誤差、角度許容誤差から構成されます。3D Viewer 上では、図16の緑色の球体と黄色の矢印からなり、球体部分を選択すると、青色の半円「ギズモ」が表示されます。すると、画面左に「選択中のウェイポイント」というウィジェットが表示され、waypoint に関する編集が可能となります。

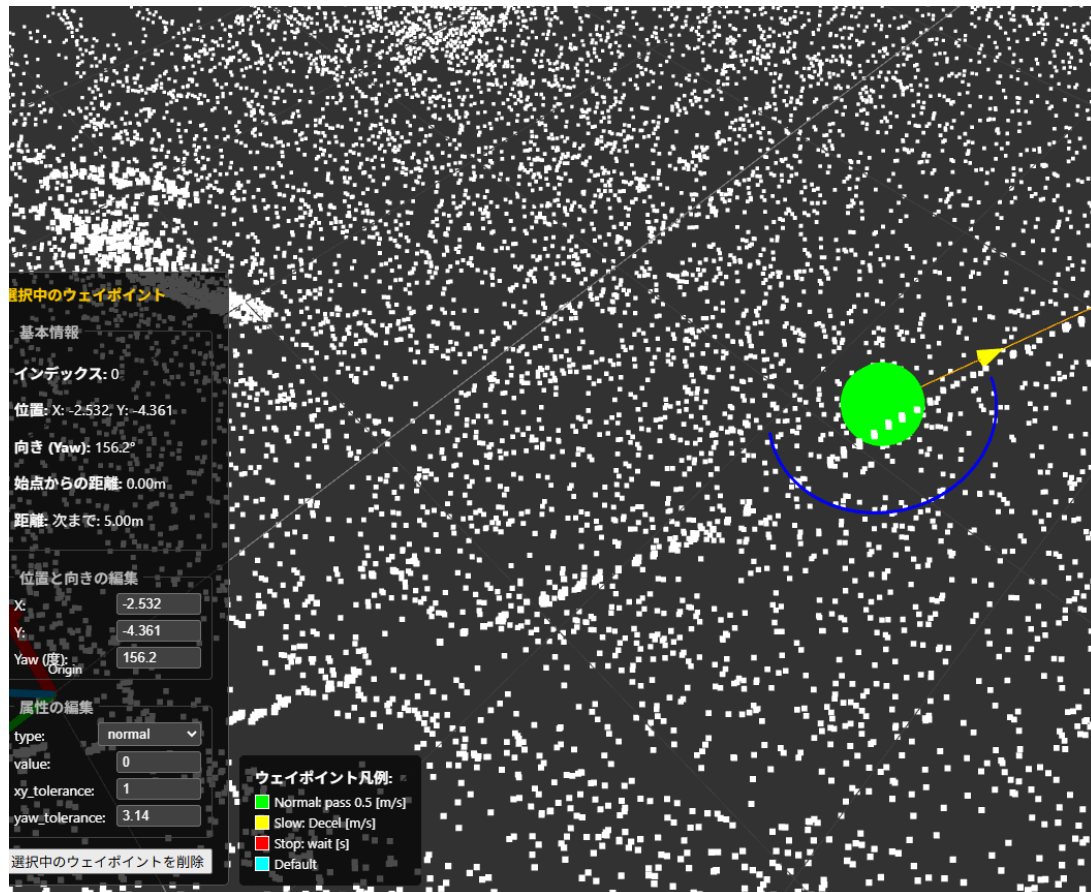


図 16: 3D Viewer で waypoint を選択した場合の画面

7.1 経路の形式

waypoint ファイルは json 形式で、下記の通りです。

```
[
  [position_x, position_y, position_z],
  [orientation_x, orientation_y, orientation_z, orientation_w],
  {
    "type": "normal",      // 属性タイプ ("normal", "stop", "slow")
    "value": 0,           // 属性の値 (停止時間[s] や 制限速度[m/s])
    "xy_tolerance": 0.25, // 到着判定の距離許容誤差 [m]
    "yaw_tolerance": 0.25 // 到着判定の角度許容誤差 [rad]
  }
]
```

position: 2D 地図上の 2 次元の位置です。z 座標は 0.0 です。x, y, z は、3D Viewer 上の origin を原点とした相対座標です。原点と緯度経度高度の対応付けがされています。

orientation: 2D 地図上の 2 次元の姿勢です。orientation_x と orientation_y は 0.0 です。

type: waypoint 到着時の制御パターン(属性)です。属性は、以下 3 種類あります。

- normal: 通過
- stop: 指定秒数停止
- slow: 次の waypoint まで指定速度まで減速します。

value: 属性の値、停止時間 [s] や 制限速度 [m/s] です。

xy_tolerance: 到着判定の距離許容誤差 [m] です。

yaw_tolerance: 到着判定の角度許容誤差 [rad] です。

7.2 3D Viewer での編集方法

「メイン画面」から「Map Viewer」をクリックし、[図 14 \(b\)](#) の画面を開きます。

1. [図 14 \(b\)](#) の状態から「ウェイポイントファイル」「PCD ファイル」のプルダウンからそれぞれファイルを選び、編集したい waypoint ファイルと、周囲の環境を確認するための 3D 点群地図を選択します。
2. [図 15](#) のように 3D 点群地図と waypoint が表示されたら、waypoint を選択し、[図 16](#) に示す、「選択中のウェイポイント」ウィジェットと、 Gizmo が表示されているか確認して下さい。

7.3 3D Viewer の基本操作

7.3.1 ファイルの読み込み

[図 17](#) の「Grid & Map Settings」から行います。

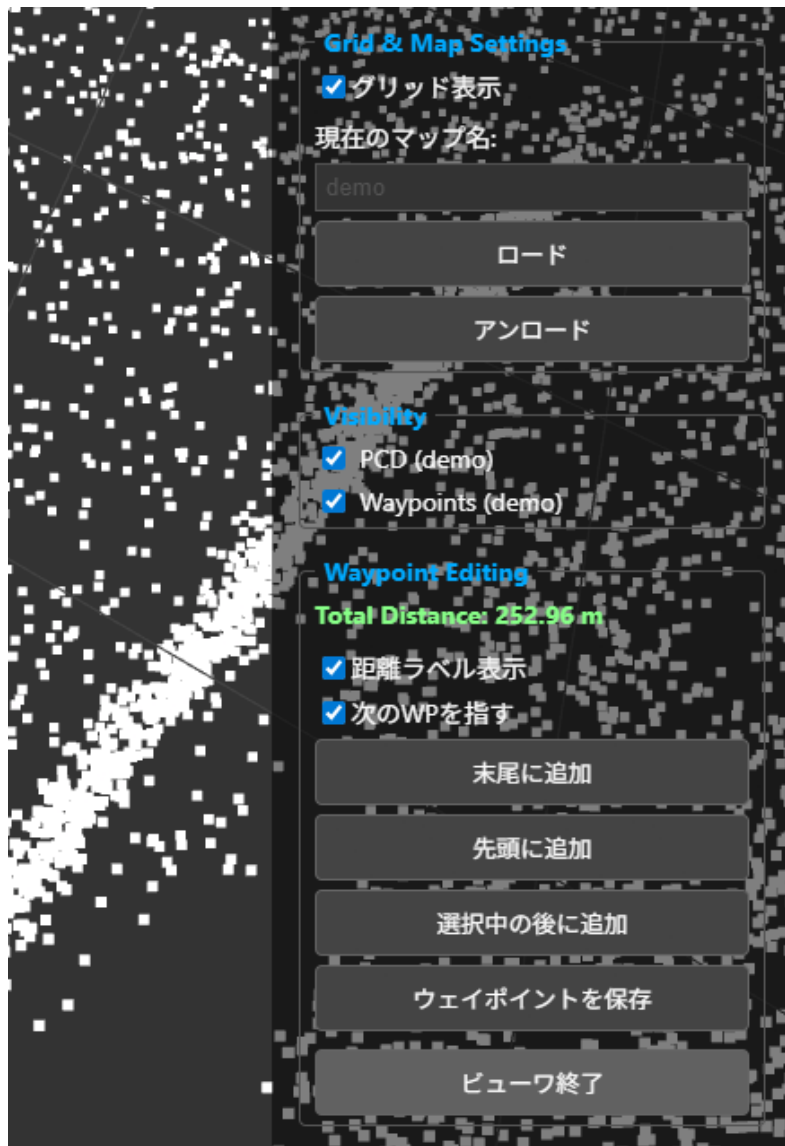


図 17: 基本編集画面

ロード: PCD、PGM、waypoint 形式の json ファイルを読み込みます。これら 3 種類のファイルを同時に読み込み重ねて編集できます。各種類に対して一つのファイルが編集可能です。

アンロード: 図 17 に示す、「アンロード」ボタンをクリックすると読み込んだファイルを閉じることができます。「アンロード」ボタンをクリックした後に、図 18 の画面で、チェックリストを外して、「選択した要素をアンロード」をクリックすると、チェックを外したファイルを閉じます。



図 18: アンロード

7.3.2 選択・回転・移動

waypoint の球体部分を選択したまま画面をドラッグすると waypoint の位置を移動することができます。ギズモは waypoint の 2D 姿勢調整用の回転ハンドルとして使用します。ギズモの線を選択したまま画面をドラッグすると、waypoint の姿勢を変更することができます。

7.3.3 属性設定

図 21 (b) の「選択中のウェイポイント」ウィジェットからは、位置と向きの編集と、属性の編集が可能です。位置と向きは、waypoint の基本操作を行うとそちらが上書きされます。位置と向きの編集は、テキストボックスとなっているため、必要な数値を入力してください。位置は、origin を基準座標とした相対座標で入力してください。デフォルトの設定では、waypoint 同士の間隔が表示されるため、こちらもご参考下さい。yaw に関しては、-180 ～ 180 の範囲で入力してください。

7.3.4 追加・削除・保存

waypoint の追加・削除は、図 17 に示す基本編集画面から行います。

追加: 図 17 Waypoint Editing から編集します。

- 末尾に追加：連番の最後の waypoint を追加します。
- 先頭に追加：連番の最初の waypoint を追加します。
- 選択中の後に追加：waypoint を選択すると有効化されます。選択した waypoint の次の連番に追加されます。

同様の操作のため、例として「選択中の後に追加」を説明します。

1. 「選択中の後に追加」ボタンをクリックします。
2. 図 19 ボタンがオレンジ色に変わり、地図中の平面上をクリックすると waypoint が設置されます。キーボードの esc を押すか、オレンジ色のボタンのどれかを再度クリックすると waypoint を追加する操作をキャンセルできます。
3. waypoint 設置後は、ボタンが元の色に戻り、再度追加可能になります。

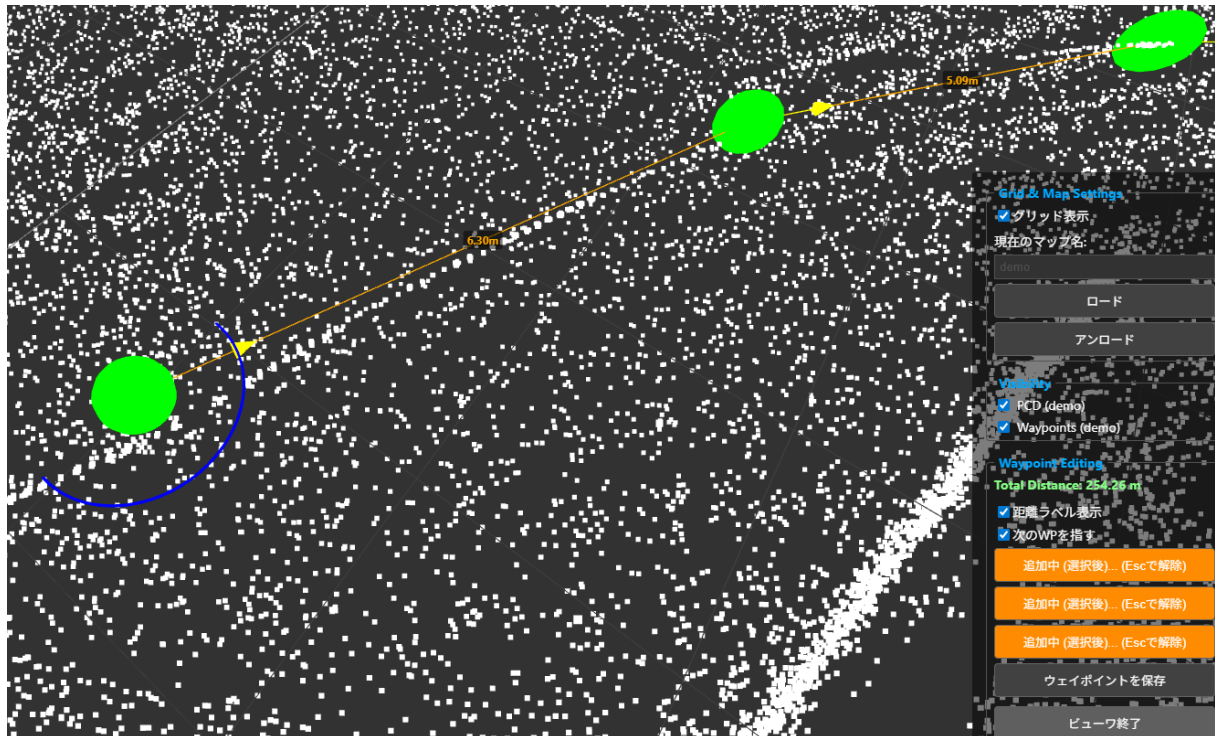


図 19: waypoint の追加押下後の画面

削除: waypoint を削除します。

1. waypoint を選択します。
2. 「選択中のウェイポイントを削除」ボタンをクリックします。
3. 図 20 に示すように、ブラウザにポップアップが表示されるため、OK を押すと waypoint が削除されます。

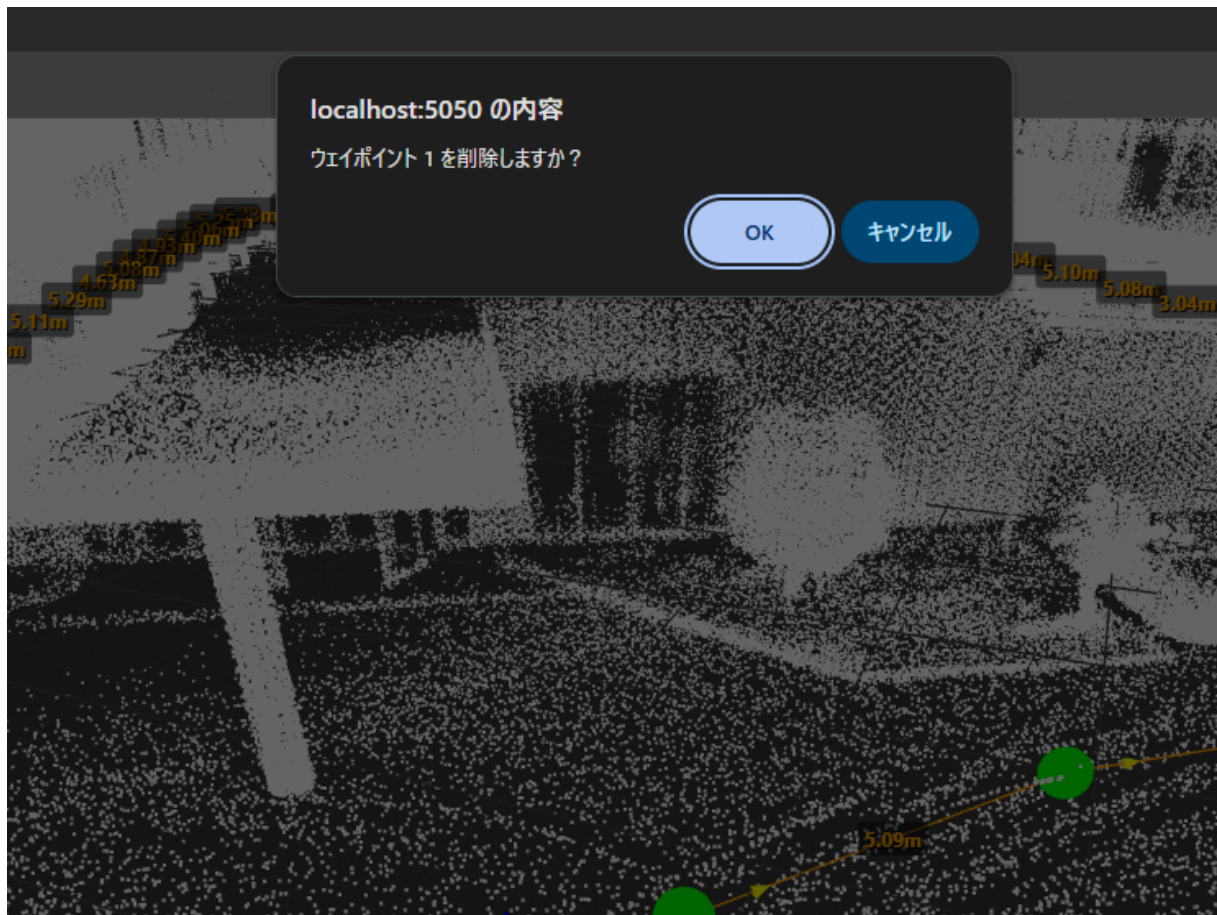
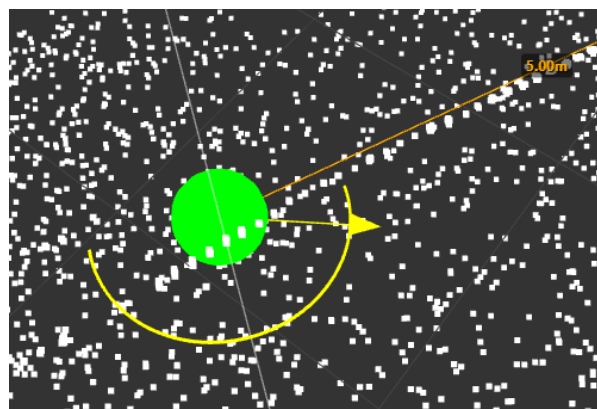


図 20: waypoint の削除押下後の画面

保存: 編集中の waypoint を保存します。

ビューワ終了: 3D Viewer を終了します。waypoint は編集時に自動で保存されますが、万が一の場合に備えて、このボタンを押す前に必ず「ウェイポイントを保存」ボタンをクリックして waypoint を保存してください。



(a) 選択時のギズモ(回転状態)



(b) 「選択中のウェイポイント」ウィジェット

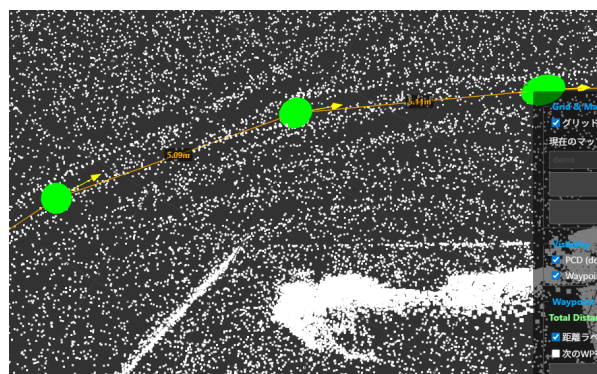
図 21: waypoint の属性

7.3.5 便利機能

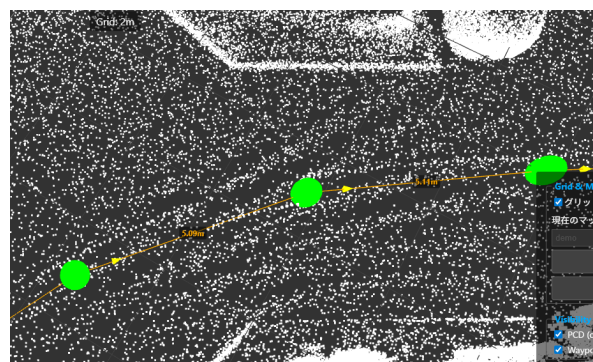
「Waypoint Editing」のその他の機能について説明します。

次の WP を指す: 図 22 (a) の状態でリストにチェックを入れると、図 22 (b) のように、すべての waypoint が次の連番の waypoint の座標に向くように一括で姿勢を調整します。リストにチェックを入れたまま waypoint を位置を調整しても、次の連番の waypoint への方向を保ったまま、姿勢が自動で調整されます。

距離ラベル表示: リストにチェックを入れるとすべての waypoint が次の連番の waypoint との距離ラベルを表示します。



(a) 「次の WP を指す前」有効化前



(b) 「次の WP を指す」有効化後

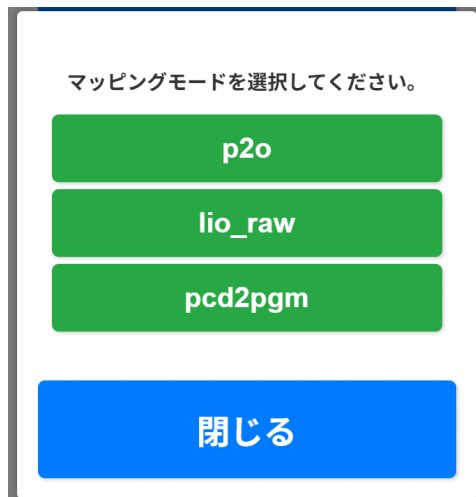
図 22: waypoint の一括向き修正

8 2D 地図への変換

静的障害物の有無の情報のために 2D 地図を作成します。3D 地図の z 方向に範囲を設けてその間の点群を全て集めて平面化します。この際、2D 地図におけるロボットの通行可能領域は waypoint によって決定されます。3D 地図を 2D 地図に変換する際、waypoint の位置と姿勢を使って、waypoint 上の静的障害物を除去することができます。waypoint 上の静的障害物を除去するためには、waypoint の位置を中心とした waypoint 上の静的障害物の除去半径をパラメータとして設定します。waypoint に対して通行可能領域の設定範囲、 z 方向の下限と上限、通行可能領域はパラメータとしています。このパラメータは、4 [コンフィグファイルの設定](#) のコンフィグファイルでパラメータを設定可能です。注意として 3D 地図に対応する 2D 地図の、拡張子以外は同じファイル名にしてください。

8.1 2D 地図への変換方法

1. メイン画面の「マッピング」をクリックし、[図 23 \(a\)](#) の「pcd2pgm」ボタンをクリックします。
2. [図 23 \(b\)](#) で 6 [マッピング](#) で作成した 3D 点群地図の PCD ファイルを選択して「選択して次へ」をクリックします。
3. [図 23 \(c\)](#) で 6 [マッピング](#) で作成した waypoint の選択画面が表示されます。waypoint を選択して「選択して実行」をクリックすると、選択した waypoint 上の静的障害物を 2D 地図から除去します。選択せずに、「スキップ(なしで続行)」を選択すると、waypoint 上の静的障害物の除去は行われません。しかし、地図作成コンフィグで使用するパラメータの内 `thre_z_min`, `thre_z_max`, `thres_point_count`, `thre_radius` を調整することで、3D 点群の高さ方向で切り出す範囲と、ノイズを除去する半径を調整することができます。これらは、使用する環境の状況に応じて使い分けてご使用下さい。
4. [図 23 \(d\)](#) に示すように、waypoint 選択した場合、ウェイポイントをループとして扱うかにチェックを入れると、地図が周回している場合最初と最後の waypoint を繋げて、障害物の除去を行います。地図が周回しない場合は、チェックを外してください。
5. [図 23 \(e\)](#) の「3D ビューアで確認」ボタンをクリックすると、3D Viewer が表示され、前述の手順で PGM ファイルを指定すると [図 23 \(f\)](#) に示すように 2D 地図が表示されます。この際、静的障害物の占有格子は赤色、通行可能領域の占有格子は緑色としています。



(a) マッピングモードクリック後の画面

PCDファイル: **demo.pcd**
 マップのPGM化と同時にウェイポイント情報 (JSON) をYAMLに含め
 さい。

利用可能なウェイポイントファイル (.json)

toyonaka_rsf.json
2025-05-20-23-56-toyonaka_rsf.json
toyonaka_test_out_input_processed.json
demo.json
2025-05-20-23-56-toyonaka_rsf_adjust.json
toyonaka_test_out_input.json

選択して続行 スキップ (なしで続行) キャンセル

(c) waypoint の選択画面

出力PGMマップベース名:

demo

(例: 'demo' → 'demo.pgm' と 'demo.yaml' が作成されます)

☒ ウェイポイントをループとして扱う (自動走行の終点から始点に戻る)

パラメータ設定ファイル (CSV):

hokuyo_slam_topics_cfg.csv

変換開始 キャンセル

PCD to PGM 変換処理が完了しました。マップファイルをダウンロードで
 さい。

☒ 変換完了!

3Dビューアで確認 PGM/YAMLマップをダウンロード

他のPCDを選択メインGUIに戻る

(e) 2D 地図への変換後の画面

利用可能なPCDファイル (.pcd)

2025-05-20-23-56-toyonaka_rsf_.pcd
2025-05-20-23-56-toyonaka_rsf_adjust.pcd
2025-05-20-23-56-toyonaka_rsf_processed.pcd
demo.pcd
toyonaka_rsf.pcd
toyonaka_test_out_input.pcd

選択して次へ 戻る

(b) 3D 点群地図の選択画面

変換元PCDファイル: **demo.pcd**

ウェイポイントファイル: **demo.json** (PCD to PGMIに反映)

出力PGMマップベース名:

demo

(例: 'demo' → 'demo.pgm' と 'demo.yaml' が作成されます)

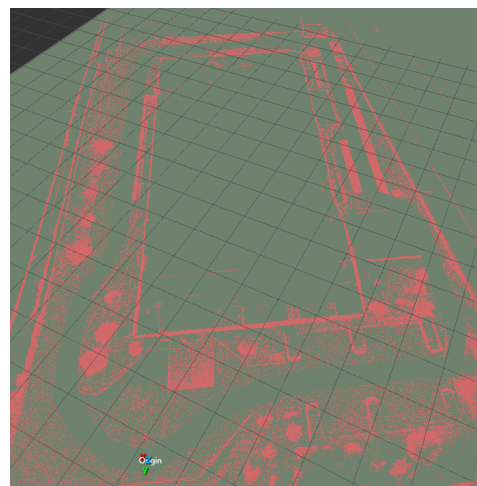
☐ ウェイポイントをループとして扱う (自動走行の終点から始点に戻る)

パラメータ設定ファイル (CSV):

hokuyo_slam_topics_cfg.csv

変換開始 キャンセル

(d) 2D 地図への変換前の画面



(f) 2D 地図の 3D Viewer 画面

図 23: 2D 地図への変換画面

8.2 平面化の調整

1. 天井が 2D 地図に投影されてしまう場合、z 方向の上限を下げることで、天井を除去できます。
2. 地面が 2D 地図に投影されてしまう場合、z 方向の下限を上げることで、地面を除去できます。

8.3 通行経路の再設定

1. 2D 地図の通行可能領域は、平面化の再実行時に waypoint を使って上書きされます。
2. 3D 点群地図と 2D の地図を重ねて表示させて通行経路を再設定し、再度 2D 地図への変換を行うと、再設定した経路に対して通行可能領域が設定されます。

A 問い合わせ先

お困りの点がございましたら、下記連絡先へお問い合わせください。

- 問い合わせ先： f-wada@hokuyo-aut.co.jp